

Integración de Firebase en la aplicación móvil Café del Centro con React Native

Cristian Arturo Hernández Flores (HF252375)

Universidad Don Bosco

Diseño y Programación de Software Multiplataforma

Ing. Alexander Alberto Sigüenza Campos

8 de noviembre de 2026

Índice

Introducción 4

Objetivos 5

Objetivo general 5

Objetivos específicos 5

Firebase en el proyecto 6

¿Qué es Firebase? 6

Por qué Firebase reemplaza a la API anterior 6

Configuración de Firebase en React Native 7

Creación del proyecto 7

Inicialización del SDK 7

Development build para el inicio con Google 7

Autenticación con Firebase 9

Registro e inicio de sesión con correo 9

Inicio de sesión con Google 9

Manejo de la sesión 10

Persistencia de datos en Cloud Firestore 12

Modelo de datos 12

Carga del catálogo 12

Creación de pedidos con transacciones 13

Sincronización en tiempo real 14

Reglas de seguridad 15

Decisiones técnicas 16

SDK de JavaScript en lugar de la librería nativa 16

Cambios concentrados en la capa de servicios 16

Limitaciones y trabajo futuro 17

Conclusiones 18

Referencias 19

Introducción

En el primer avance de la versión móvil de Café del Centro se construyeron las pantallas principales con React Native y se conectó la aplicación a la API de la versión web. Esa API guardaba los usuarios, los pedidos y los puntos en la memoria del servidor, por lo que la información se perdía cada vez que este se reiniciaba y, en Vercel, los pedidos podían aparecer en una consulta y desaparecer en la siguiente.

Este segundo avance resuelve ese problema con Firebase. El inicio de sesión pasó a Firebase Authentication, con acceso por correo electrónico y por cuenta de Google, y los datos del sistema se guardan ahora en Cloud Firestore, asociados al usuario que inició sesión. Además, el historial de pedidos y los puntos se sincronizan en tiempo real, y la base de datos está protegida con reglas de seguridad propias.

El documento explica cómo se configuró Firebase en el proyecto, cómo funciona el flujo de autenticación, cómo se organizó el modelo de datos y qué reglas lo protegen. También describe las decisiones técnicas tomadas durante la integración, las limitaciones que todavía existen y el trabajo pendiente para la entrega final.

Objetivos

Objetivo general

Integrar Firebase Authentication y Cloud Firestore en la aplicación móvil de Café del Centro para que los clientes inicien sesión de forma segura y sus pedidos, puntos e historial se guarden en una base de datos en la nube.

Objetivos específicos

- Implementar el registro y el inicio de sesión con correo electrónico y con cuenta de Google.
- Almacenar los productos, los pedidos y los puntos en Cloud Firestore, asociados al usuario autenticado.
- Mostrar el historial de pedidos y los puntos actualizados en tiempo real.
- Proteger la base de datos con reglas de seguridad que limiten lo que cada usuario puede leer y modificar.

Firebase en el proyecto

¿Qué es Firebase?

Firebase es una plataforma de Google que ofrece servicios en la nube para aplicaciones web y móviles, como autenticación de usuarios, bases de datos, almacenamiento de archivos y alojamiento (Firebase, s. f.). Su principal ventaja es que la aplicación se comunica directamente con estos servicios por medio de un SDK, sin necesidad de programar ni mantener un servidor propio.

En este avance se utilizaron dos de sus servicios. Firebase Authentication gestiona las cuentas de los usuarios y los distintos métodos de inicio de sesión, mientras que Cloud Firestore es una base de datos NoSQL que organiza la información en colecciones y documentos. El proyecto funciona dentro del plan gratuito Spark, que cubre de sobra el volumen de una aplicación en desarrollo.

Por qué Firebase reemplaza a la API anterior

La decisión de usar Firebase respondió directamente al problema detectado en el avance anterior. La API web guardaba los datos en la memoria de cada proceso del servidor, y en Vercel cada ruta podía ejecutarse en instancias distintas que no compartían información. Con Firestore, todas las consultas leen y escriben en la misma base de datos, de modo que un pedido creado desde el teléfono queda guardado de forma permanente y es el mismo para cualquier consulta posterior.

Además, Firebase ofrece el inicio de sesión con Google ya resuelto y permite proteger los datos con reglas de seguridad que se ejecutan en sus servidores. Construir estas funciones desde cero en la API propia habría requerido mucho más tiempo y habría sido menos seguro.

Configuración de Firebase en React Native

Creación del proyecto

El primer paso fue crear el proyecto en la consola de Firebase y habilitar los dos métodos de inicio de sesión que pide el avance: correo electrónico con contraseña y Google. Después se creó la base de datos de Cloud Firestore en modo de producción, que bloquea cualquier lectura o escritura hasta que se definan reglas de seguridad, y se eligió una ubicación en Estados Unidos por su cercanía con El Salvador.

Dentro del proyecto se registraron dos aplicaciones. La aplicación web proporcionó la configuración que usa el SDK de JavaScript para conectarse a Firebase, y la aplicación Android, registrada con el identificador de paquete com.codehub.cafedelcentro, permitió habilitar el inicio de sesión con Google en el teléfono.

Inicialización del SDK

La conexión con Firebase se concentra en un solo archivo de la capa de servicios, firebase.ts. Ahí se inicializa la aplicación, se prepara el servicio de autenticación y se obtiene la referencia a la base de datos. Antes de inicializar, el código revisa si Firebase ya estaba activo, porque la recarga rápida de Expo vuelve a ejecutar los archivos al guardar cambios y, sin esa revisión, la aplicación intentaría inicializarse dos veces.

Para la autenticación se configuró la persistencia con AsyncStorage (React Native Async Storage, s. f.). Así, Firebase guarda la sesión en el teléfono y el cliente no tiene que volver a iniciar sesión cada vez que abre la aplicación. Con esto se reemplazó el manejo manual de la sesión que se había programado en el avance anterior.

Development build para el inicio con Google

El inicio de sesión con correo funciona directamente en Expo Go, pero el de Google no. Para mostrar la ventana nativa de selección de cuentas se usa la librería oficial de Google Sign In para React Native, que incluye código nativo de Android (React Native Google Sign

In, s. f.). Como Expo Go solo trae un conjunto fijo de librerías nativas, fue necesario generar una development build, es decir, una versión instalable de la propia aplicación que incluye esas librerías (Expo, s. f.).

La build se compiló en la nube con EAS, el servicio de compilación de Expo, que también generó la llave con la que se firma la aplicación. La huella SHA1 de esa llave se registró en la aplicación Android de Firebase, porque Google solo permite iniciar sesión desde aplicaciones cuya firma conoce. Una vez instalada la build en el emulador, el desarrollo continuó igual que antes: los cambios en el código se cargan al instante, sin volver a compilar.

Autenticación con Firebase

Registro e inicio de sesión con correo

Cuando un cliente se registra, la aplicación crea su cuenta en Firebase Authentication, guarda su nombre en el perfil de la cuenta y crea un documento con sus datos en la colección usuarios de Firestore. Ese documento usa como identificador el mismo UID que Firebase asigna a la cuenta, lo que permite relacionar sin ambigüedad la cuenta con su perfil y, más adelante, con sus pedidos.

Las validaciones del avance anterior se mantuvieron, de modo que los datos se revisan en el teléfono antes de enviarse. Firebase agrega una segunda capa: rechaza correos ya registrados, contraseñas débiles o credenciales incorrectas, y responde con un código de error. Esos códigos se traducen a mensajes claros en español, como “Correo o contraseña incorrectos”, para que el cliente entienda qué ocurrió.

Inicio de sesión con Google

El inicio con Google sigue un flujo de cuatro pasos. Primero, la aplicación verifica que el teléfono tenga Google Play Services y abre la ventana nativa para elegir una cuenta. Segundo, Google devuelve un token de identidad que confirma quién es el usuario. Tercero, ese token se convierte en una credencial de Firebase y se usa para iniciar sesión. Por último, si es la primera vez que esa cuenta entra, se crea su perfil en Firestore con el nombre y el correo de Google y con cero puntos.

Si el cliente cierra la ventana sin elegir una cuenta, la aplicación vuelve al inicio de sesión sin mostrar ningún error. Al cerrar sesión, también se cierra la sesión de Google, para que la próxima vez la aplicación vuelva a preguntar con qué cuenta entrar. La Tabla 1 compara los dos métodos y la Figura 1 muestra la pantalla de inicio de sesión.

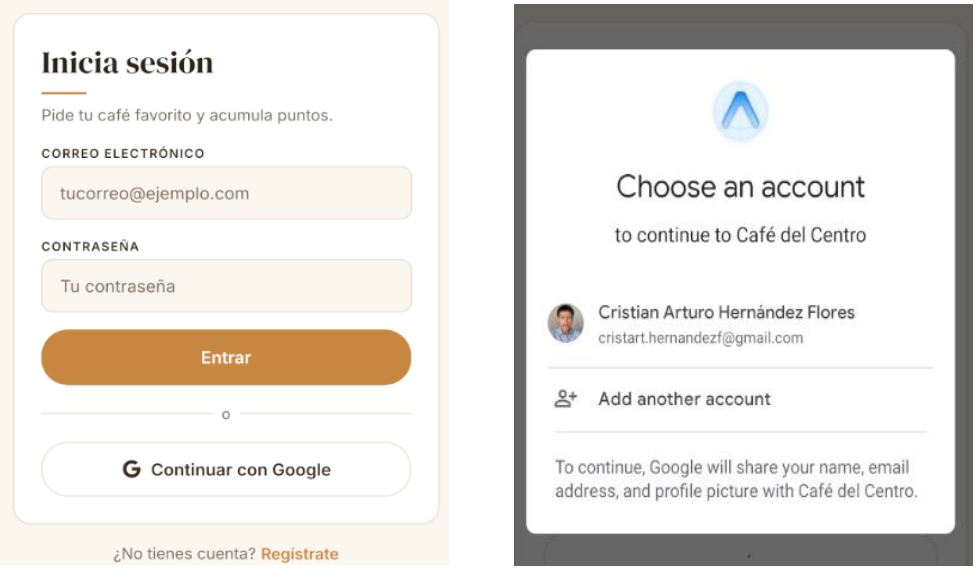
Tabla 1

Métodos de inicio de sesión

Aspecto	Correo electrónico	Google
Creación de la cuenta	Formulario de registro de la aplicación	Automática en el primer inicio de sesión
Verificación de identidad	Contraseña de al menos 6 caracteres	Cuenta y verificación de Google
Perfil en Firestore	Se crea al registrarse	Se crea al iniciar sesión por primera vez
Requisito técnico	Funciona en Expo Go	Requiere la development build y la huella SHA1

Figura 1

Pantalla de inicio de sesión con el acceso por Google



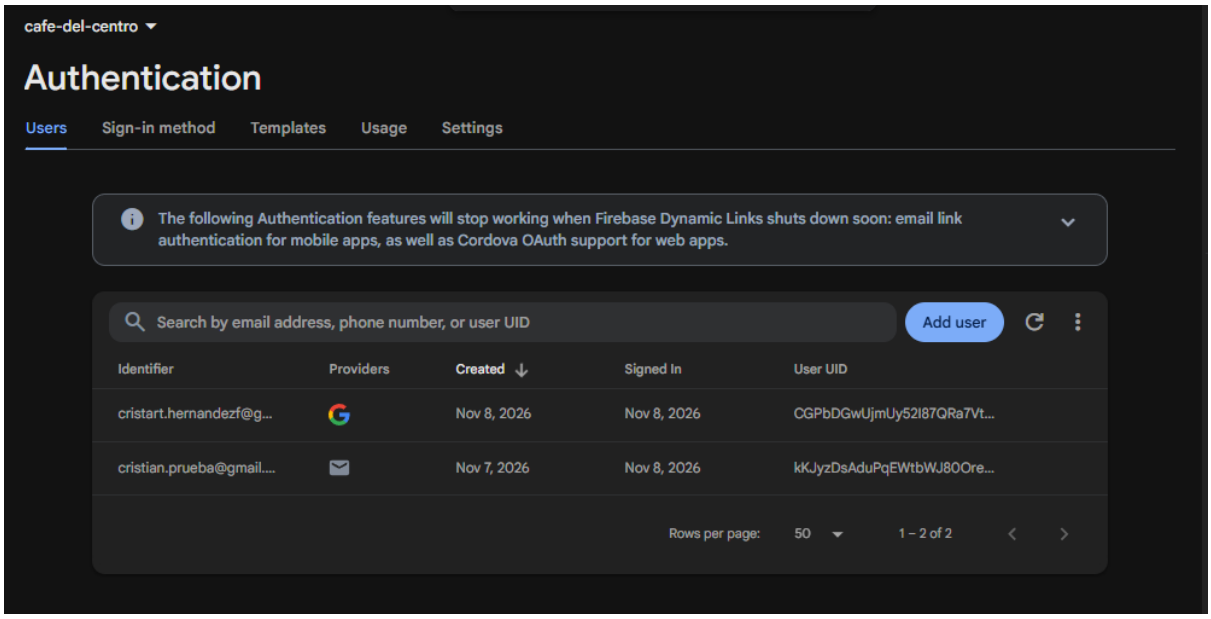
Manejo de la sesión

La aplicación escucha en todo momento el estado de la sesión por medio de Firebase Authentication. Cuando detecta un usuario, carga su perfil desde Firestore y permite el acceso a las pestañas principales; cuando no lo hay, lo redirige a la pantalla de inicio de sesión. Esta escucha también actúa al abrir la aplicación: si la sesión estaba guardada, el cliente entra directamente al inicio.

La regla de que la aplicación es solo para clientes se mantuvo. Si una cuenta tiene el rol de administrador, la sesión se cierra de inmediato y se muestra un mensaje que indica que el panel de administración está en la versión web. La Figura 2 muestra las dos cuentas de prueba registradas en Firebase, una con cada método.

Figura 2

Usuarios registrados en Firebase Authentication



Persistencia de datos en Cloud Firestore

Modelo de datos

Firestore organiza la información en colecciones, que agrupan documentos, y cada documento guarda sus datos como pares de campo y valor. Para el proyecto se definieron tres colecciones, que se describen en la Tabla 2.

Tabla 2

Modelo de datos en Cloud Firestore

Colección	Identificador	Campos principales	Relación
usuarios	UID de la cuenta	nombre, email, rol, puntos y fecha de creación	Un documento por cada cuenta de Authentication
productos	Número del producto	nombre, descripción, categoría, imagen, precios por tamaño, orden y disponibilidad	Catálogo compartido por todos los clientes
pedidos	Generado por Firestore	usuarioId, artículos, total, estado, método y referencia de pago, y fecha	Cada pedido se asocia a su dueño por medio de usuarioId

La asociación con el usuario autenticado se resuelve con el campo usuarioId de cada pedido, que guarda el UID de quien lo hizo. Así, para mostrar el historial basta con consultar los pedidos cuyo usuarioId coincide con el usuario de la sesión. Los artículos se guardan dentro del mismo pedido, con el nombre, el tamaño, el precio y la cantidad de cada bebida, para que el historial muestre lo que se pidió aunque el catálogo cambie después.

Carga del catálogo

Los seis productos del menú se cargaron una sola vez por medio de un script de Node.js que escribe cada producto en la colección productos, con las mismas fotografías y precios de la versión web. Mientras se ejecutó el script, las reglas permitieron la escritura en esa colección; después se cerraron para que el catálogo no pueda modificarse desde la

aplicación. Cada producto incluye un campo orden, que permite mostrarlos siempre en la misma secuencia.

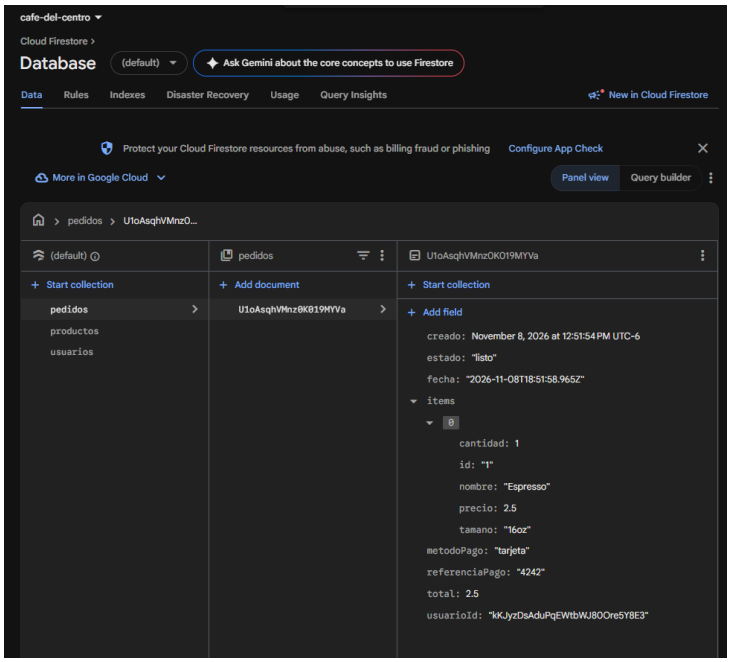
Creación de pedidos con transacciones

Confirmar un pedido implica dos escrituras: crear el documento del pedido y sumar los puntos al perfil del cliente. Si solo una de ellas se completara, por ejemplo por una falla de conexión, el cliente podría quedar con un pedido sin puntos o con puntos sin pedido. Para evitarlo, ambas operaciones se ejecutan dentro de una transacción de Firestore, que aplica las dos o ninguna.

Dentro de la transacción se leen los puntos actuales del cliente, se calcula el nuevo total a razón de un punto por cada dólar y se guardan el pedido y los puntos al mismo tiempo. El pedido almacena la fecha en dos formatos: una fecha en texto, que la aplicación usa para ordenar y mostrar el historial, y una marca de tiempo generada por el servidor de Firebase, que no depende del reloj del teléfono. La Figura 3 muestra un pedido tal como queda guardado en la base de datos.

Figura 3

Pedido almacenado en Cloud Firestore



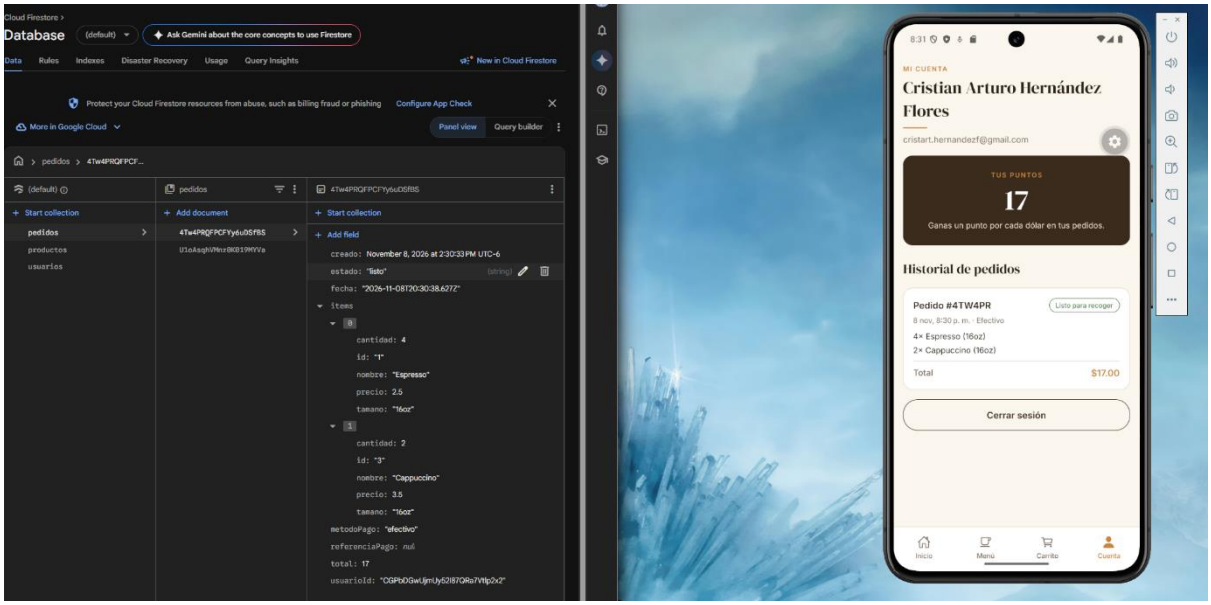
Sincronización en tiempo real

En el avance anterior, la pantalla de Cuenta consultaba la API cada cinco segundos para detectar cambios en los pedidos. Con Firestore se reemplazó esa consulta repetida por escuchas en tiempo real: la aplicación se suscribe al perfil del cliente y a sus pedidos, y Firestore le avisa en cuanto algo cambia.

Gracias a esto, los puntos se actualizan en el instante en que se confirma un pedido y el historial refleja cualquier cambio de estado sin que el cliente tenga que refrescar la pantalla. Durante las pruebas, al cambiar el estado de un pedido de pendiente a listo desde la consola de Firebase, la aplicación lo mostró como “Listo para recoger” en menos de un segundo, como se observa en la Figura 4.

Figura 4

Cambio de estado de un pedido en tiempo real



Para el historial, la consulta filtra los pedidos por usuarioId y el orden por fecha se aplica en el teléfono. De esta forma no fue necesario crear un índice compuesto en Firestore, algo innecesario para la cantidad de pedidos que tiene cada cliente. Además, como los identificadores que genera Firestore son largos, la aplicación muestra solo sus primeros seis caracteres como código del pedido.

Reglas de seguridad

Como la aplicación se comunica directamente con Firestore, las reglas de seguridad son la barrera entre los datos y cualquier persona que intente acceder a ellos. Firebase las evalúa en sus servidores antes de cada lectura o escritura, por lo que no pueden evitarse modificando la aplicación (Firebase, s. f.). La base de datos se creó en modo de producción, que lo bloquea todo, y sobre esa base se abrieron solo los permisos necesarios, como resume la Tabla 3.

Tabla 3

Reglas de seguridad por colección

Colección	Lectura	Creación	Modificación y eliminación
usuarios	Solo el propio perfil	Solo el propio perfil, con rol de cliente y cero puntos	Solo el propio perfil, sin cambiar el rol ni reducir los puntos
productos	Cualquier usuario con sesión iniciada	No permitida	No permitida
pedidos	Solo los pedidos propios	Solo a nombre propio, en estado pendiente, con total mayor a cero y al menos un artículo	No permitida

Con estas reglas, un cliente no puede ver los pedidos ni el perfil de otra persona, no puede crear una cuenta con rol de administrador, no puede registrar pedidos a nombre de otro usuario ni marcar su propio pedido como entregado, y no puede modificar el catálogo. Las reglas se probaron desde la aplicación: el registro, la compra y el historial funcionan con normalidad, mientras que cualquier operación fuera de lo permitido devuelve un error de permisos que la aplicación muestra como “No tienes permiso para realizar esta acción”.

Decisiones técnicas

SDK de JavaScript en lugar de la librería nativa

Para usar Firebase en React Native existen dos opciones: el SDK oficial de JavaScript y una librería que envuelve los SDK nativos de Android e iOS. Se eligió el SDK de JavaScript porque funciona en Expo Go, lo que permitió integrar la autenticación por correo y Firestore sin compilar la aplicación, y porque es el mismo SDK que podrá usarse en la versión web (Expo, s. f.). La única parte que necesitó código nativo fue la ventana de Google, que se resolvió con su librería oficial.

Cambios concentrados en la capa de servicios

La arquitectura en capas planteada en el avance anterior cumplió su propósito. Las pantallas siguen llamando a las mismas funciones para iniciar sesión, obtener el menú o crear un pedido; lo que cambió es lo que esas funciones hacen internamente. La Tabla 4 resume esos cambios.

Tabla 4

Cambios en la capa de servicios entre ambos avances

Función	Primer avance	Segundo avance
Registro e inicio de sesión	Petición a la API web	Firebase Authentication con correo y Google
Sesión guardada	Datos del usuario en AsyncStorage	Persistencia de Firebase Authentication
Menú	Petición a la API web	Consulta a la colección productos
Crear pedido	Petición a la API web	Transacción en Firestore
Historial y puntos	Consulta cada cinco segundos	Escucha en tiempo real

Las pantallas de menú, carrito y pago no necesitaron cambios en su diseño, lo que confirma que separar la lógica de acceso a datos de la interfaz fue una buena decisión desde el inicio del proyecto.

Limitaciones y trabajo futuro

La versión web todavía usa la API con datos en memoria, por lo que el panel de administración no ve los pedidos hechos desde el teléfono. En este avance, el cambio de estado de los pedidos se probó desde la consola de Firebase. Para la entrega final, la versión web se conectará a la misma base de datos de Firestore, de modo que ambas versiones compartan el catálogo, los pedidos y los puntos.

Para que el administrador pueda cambiar el estado de los pedidos desde la web, será necesario ampliar las reglas de seguridad con un permiso específico para su rol, ya que actualmente ningún usuario puede modificar un pedido una vez creado.

Las reglas impiden que un cliente reduzca sus puntos o cambie su rol, pero no verifican que los puntos sumados correspondan exactamente al total del pedido. La forma más segura de resolverlo sería calcular los puntos en el servidor con Cloud Functions, aunque esa opción requiere el plan de pago de Firebase.

Por último, el inicio de sesión con Google está configurado solo para Android. Para usarlo en iOS habría que registrar una aplicación de ese sistema en Firebase y generar una build propia.

Conclusiones

Firebase resolvió el principal problema del avance anterior. Al pasar de una API con datos en memoria a una base de datos en la nube, los pedidos y los puntos dejaron de perderse o mostrarse de forma inconsistente, y la aplicación presenta siempre la misma información, sin importar cuántas veces se consulte.

La integración también dejó claro que trabajar sin un servidor propio cambia la forma de pensar la seguridad. Como el teléfono se comunica directamente con la base de datos, las validaciones de la aplicación sirven para guiar al usuario, pero la protección real está en las reglas de Firestore. Diseñarlas obligó a definir con precisión qué puede hacer cada usuario con cada colección.

El inicio de sesión con Google fue la parte más laboriosa, no por el código, sino por la configuración: la development build, la firma de la aplicación y su registro en Firebase. Fue una muestra clara de la diferencia entre probar una aplicación en Expo Go y prepararla para funcionar como una aplicación real.

Por último, la arquitectura en capas definida desde el primer avance permitió que la integración se concentrara en la capa de servicios y en el contexto de sesión, sin rehacer las pantallas. Con este avance, Café del Centro cuenta con una aplicación móvil con autenticación segura, datos persistentes y sincronización en tiempo real, preparada para conectarse con la versión web en la entrega final.

Referencias

Expo. (s. f.). *Using Firebase*. Expo Documentation. https://docs.expo.dev/guides/using-firebase/

Firebase. (s. f.). *Firebase documentation*. https://firebase.google.com/docs

React Native Async Storage. (s. f.). *Usage*. https://react-native-async-storage.github.io/async-storage/docs/usage

React Native Google Sign In. (s. f.). *React Native Google Sign In documentation*. https://react-native-google-signin.github.io/docs/